MËSIMI 28 / SHËNIME PËR INSTRUKTORIN

# Kërcimi, përplasjet dhe pikët

**75 minuta • App.js, Bird.js dhe Obstacles.js**

**Rezultati i pritshëm**
Nxënësi shton kontrollin e përplasjes, funksionin jump dhe rezultatin score. Prekja rrit birdBottom me 50, si te kodi i mësimit.

## Rrjedha e orës

| Koha | Çfarë bën instruktori |
|---|---|
| 0-5 | Rikujtim: birdBottom, obstaclesLeft dhe gap. |
| 5-20 | Shpjego kushtin e përplasjes për një çift. |
| 20-35 | Shto gameOver dhe kontrollin për çiftin e dytë. |
| 35-50 | Mbështill View me TouchableWithoutFeedback dhe lidh jump. |
| 50-62 | Shto score në degët e rikthimit të pengesave. |
| 62-70 | Provo prekjen, përplasjen dhe numërimin. |
| 70-75 | Pyetje kontrolli dhe ruajtja e punës. |

## Si ta përdorësh këtë PDF

Tre faqet e para janë shënime në shqip për drejtimin e orës. Më pas përdor materialin origjinal dhe kodin e tij në JavaScript. Jep një hap, provo rezultatin dhe vetëm pastaj vazhdo me hapin tjetër.

Ruaj të njëjtët emra variablash dhe të njëjtat props si në fotografitë e kodit. Shënimet shpjegojnë shembullin ekzistues; nuk shtojnë funksione të reja në lojë.

MËSIMI 28 / SHËNIME PËR INSTRUKTORIN

# Përplasja dhe kërcimi në stilin origjinal

## 1. Shpjego kushtin për një çift

Ndaje kushtin e gjatë në dy pyetje: **A është pengesa afër zogut horizontalisht?** dhe **A është zogu jashtë hapësirës së lirë vertikalisht?** Vetëm kur plotësohen të dyja, kërkohet ndalimi i lojës. Pastaj përsërite për çiftin Two.

## 2. Lexo funksionin gameOver

Në burim ai ndalon intervalin e zogut dhe dy intervalet e pengesave me clearInterval. Kërko nga nxënësi të tregojë cilit interval i përket secili emër. Mos e paraqit si buton rinisjeje: moduli origjinal nuk ndërton një të tillë.

## 3. Mbaj TouchableWithoutFeedback

Ai mbështjell një View që përmban zogun, pengesat dhe tekstin. onPress={jump} thërret funksionin kur preket ekrani. Kjo API ende ekziston; nuk kërkohet ndryshim vetëm për të ndjekur këtë mësim.

## 4. Jump mbetet ndryshim i pozicionit

```
setBirdBottom(birdBottom => birdBottom + 50)
```

Meqë përdoret bottom, shtimi i vlerës e ngre zogun. Intervali i rënies vazhdon ta ulë atë. Këtu mësohet një kërcim i thjeshtë me zhvendosje, njësoj si në materialin origjinal.

**Demonstrimi**
Bëj një prekje dhe pyet çfarë ndryshoi. Pastaj mos prek për pak kohë: zogu duhet të vazhdojë rënien. Mbaj të njëjtat vlera dhe të njëjtët emra si te fotografitë e kodit.

MËSIMI 28 / SHËNIME PËR INSTRUKTORIN

# Pikët dhe prova e nxënësve

## Kur shtohet score sipas materialit

Në këtë version, +1 vendoset te dega else ku pengesa rikthehet në të djathtë. Pra rezultati ndryshon kur çifti del nga ekrani dhe riciklohet, jo saktësisht në çastin kur kalon pranë zogut. Shpjego atë që bën kodi i dhënë.

```
setScore(score => score + 1)
```

Shto rreshtin te të dy çiftet, sipas hapit të mësimit. Text shfaq vlerën e score. Mos vendos shtimin te intervali që ekzekutohet çdo 30 ms, sepse rezultati do të rritet vazhdimisht.

| Prova | Çfarë të vëzhgosh |
|---|---|
| Prek ekranin | jump thirret dhe birdBottom rritet. |
| Kalo në hapësirën mes pengesave | Zogu nuk duhet të trajtohet si jashtë hapësirës. |
| Përplasu me pengesën | Kontrollo thirrjen gameOver dhe ndalimin e intervaleve. |
| Lër një çift të dalë jashtë ekranit | score rritet një herë te rikthimi. |
| Lër edhe çiftin e dytë të dalë | Edhe dega e tij duhet të shtojë score. |

## Pyetje për vlerësim

**Çfarë thërret prekja?** jump.
**Çfarë ndalon gameOver?** Tre intervalet.
**Ku shtohet pikë në këtë material?** Te else i rikthimit të pengesës.

## Burimi dhe komandat e verifikuara

Materiali bazë: Lesson 28, Digital School, PDF-ja e furnizuar. Pas këtyre 3 faqeve vijon materiali origjinal me numërtimin e vet. Kodi dhe rendi i hapave ruhen; në modulin 26 janë përditësuar vetëm komandat e krijimit dhe nisjes.

Template blank: JavaScript me strukturë minimale
https://docs.expo.dev/more/create-expo/

Nisja e projektit me Expo CLI
https://docs.expo.dev/more/expo-cli/

TouchableWithoutFeedback
https://reactnative.dev/docs/touchablewithoutfeedback

Verifikuar më 31.10.2026. Provoje hapjen në pajisjen e klasës para orës; përputhshmëria e Expo Go varet nga SDK-ja e projektit.

Lesson 28 (Estimated time: 75 minutes)

# Lesson 28

## Jumping Behavior(Bird) & Obstacles Behavior

### Contents:

- Jump Behavior

### Notes for instructor:

Before starting to teach this lesson, you'll have to do a 5-minute review previous lesson to make sure they remember something from previous lessons. After you finish the review you should explain on the board the contents just to get them ready. All the coursework completed in class should be uploaded to GitHub.

### Jump Behavior

In today's lesson, we're going to focus on two features: the jump behavior of our Bird Component, and the other is to check for collisions. We start by creating an useEffect function on our App.js file which will be the function that checks for collisions.

```
//check for collisions
useEffect(() => {
  if (
    ((birdBottom < (obstaclesNegHeight + obstacleHeight + 30) ||
    birdBottom > (obstaclesNegHeight + obstacleHeight + gap -30)) &&
    (obstaclesLeft > screenWidth/2 -30 && obstaclesLeft < screenWidth/2 + 30 )
    )
    ||
    ((birdBottom < (obstaclesNegHeightTwo + obstacleHeight + 30) ||
    birdBottom > (obstaclesNegHeightTwo + obstacleHeight + gap -30)) &&
    (obstaclesLeftTwo > screenWidth/2 -30 && obstaclesLeftTwo < screenWidth/2 + 30
    )
    )
    {
    console.log('game over')
  }
})
```

This function will console the log game over if we touch an Obstacle and stop the movement. For this, we can create another function that is called gameOver which will stop the game after we touch any Obstacle

```
//check for collisions
useEffect(() => {
  if (
    ((birdBottom < (obstaclesNegHeight + obstacleHeight + 30) ||
    birdBottom > (obstaclesNegHeight + obstacleHeight + gap -30)) &&
    (obstaclesLeft > screenWidth/2 -30 && obstaclesLeft < screenWidth/2 + 30 )
    )
    ||
    ((birdBottom < (obstaclesNegHeightTwo + obstacleHeight + 30) ||
    birdBottom > (obstaclesNegHeightTwo + obstacleHeight + gap -30)) &&
    (obstaclesLeftTwo > screenWidth/2 -30 && obstaclesLeftTwo < screenWidth/2 + 30 )
    )
    )
    {
    console.log('game over')
    gameOver()
  }
})
```

```
//game over function
const gameOver = () => {
  clearInterval(gameTimerId)
  clearInterval(obstaclesTimerId)
  clearInterval(obstaclesTimerIdTwo)
}
```

After we're done with the gameOver function, when we run the App, we will see that the game is going to stop immediately after the Bird Component touches the Obstacle Component.

We continue by creating the jump function, and first of all, we need to import the **TouchableWithoutFeedback** element. **TouchableWithoutFeedback** supports only one child. If you wish to have several child components, wrap them in a View. Importantly, TouchableWithoutFeedback works by cloning its child and applying responder props to it. It is therefore required that any intermediary components pass through those props to the underlying React Native component.

We wrap our View element with Touchable element, so whenever the user clicks on the screen, we will call the **jump** function

```
return (
  <TouchableWithoutFeedback onPress={jump}>
    <View style={styles.container}>
      {isGameOver && <Text style={{fontSize: '30px'}}>{score}</Text>}
      <Bird
        birdBottom = {birdBottom}
        birdLeft = {birdLeft}
      />
      <Obstacles
        color={'green'}
        obstacleWidth = {obstacleWidth}
        obstacleHeight = {obstacleHeight}
        randomBottom = {obstaclesNegHeight}
        gap = {gap}
        obstaclesLeft = {obstaclesLeft}
      />
      <Obstacles
        color={'yellow'}
        obstacleWidth = {obstacleWidth}
        obstacleHeight = {obstacleHeight}
        randomBottom = {obstaclesNegHeightTwo}
        gap = {gap}
        obstaclesLeft = {obstaclesLeftTwo}
      />
    </View>
  </TouchableWithoutFeedback>
);
```

We continue to make the logic behind the **jump** function

```
//jump behaviour
const jump = () => {
 if (!isGameOver && (birdBottom < screenHeight)) {
   setBirdBottom(birdBottom => birdBottom + 50)
   console.log('jumped')
 }
}
```

After we're done with the function, we will see that everything is working well until now when we run the Game. By clicking on the screen, we can jump, and we're passing through Obstacles.

We continue by adding a score on our game, so whenever we pass an Obstacle, we will get +1 points. We declare variables and inside useEffect of our two obstacles we add this:

```
//start second obstacle
useEffect(() => {
  if (obstaclesLeftTwo > -60) {
    obstaclesTimerIdTwo = setInterval(() => {
      setObstaclesLeftTwo(obstaclesLeftTwo => obstaclesLeftTwo - 5)
    }, 30)
      return () => {
        clearInterval(obstaclesTimerIdTwo)
      }
    } else {
        setScore(score => score +1)
        setObstaclesLeftTwo(screenWidth)
        setObstaclesNegHeightTwo( - Math.random() * 100)
      }
}, [obstaclesLeftTwo])
```

Then inside App.js, we print that value using Text element

```
return (
  <TouchableWithoutFeedback onPress={jump}>
    <View style={styles.container}>
      {isGameOver && <Text style={{fontSize: '30px'}}>{score}</Text>}
      <Bird
        birdBottom = {birdBottom}
        birdLeft = {birdLeft}
      />
      <Obstacles
        color={'green'}
        obstacleWidth = {obstacleWidth}
        obstacleHeight = {obstacleHeight}
        randomBottom = {obstaclesNegHeight}
        gap = {gap}
        obstaclesLeft = {obstaclesLeft}
      />
      <Obstacles
        color={'yellow'}
        obstacleWidth = {obstacleWidth}
        obstacleHeight = {obstacleHeight}
        randomBottom = {obstaclesNegHeightTwo}
        gap = {gap}
        obstaclesLeft = {obstaclesLeftTwo}
      />
    </View>
  </TouchableWithoutFeedback>
)
```

**Source Code: Flappy Bird**

**Award the student who demonstrated expertise in testing practices with the Testing Guru of React Native badge:** Award

### Evaluation Criteria:

✓ All students should understand all of the features of Today's lesson

✓ Badges should be awarded

### Tools needed:

✓ Computers

✓ VS Code

Jump to...



**Copyright © Digital School. All rights reserved.**